Architectural Analysis and Optimization of AI Agents for Geotechnical Finite Element Modeling

The integration of artificial intelligence into specialized engineering domains has transitioned from deterministic, rule-based scripting to autonomous, agentic systems capable of dynamic decision-making.[1] In geotechnical engineering, numerical modeling software such as PLAXIS 2D and 3D serves as the industry standard for executing finite element analyses (FEA) to evaluate soil-structure interaction, deformation, stability, and groundwater flow.[3] Setting up these models, defining complex soil stratigraphy, and post-processing high-dimensional output data remain highly manual, time-consuming, and error-prone tasks.[6]

An artificial intelligence agent designed for PLAXIS—such as the conceptualized plaxis-agent or similar emerging frameworks—bridges the gap between high-level human intent and the rigid technical execution required by geomechanical software.[8] While specific private repositories like souga15/plaxis-agent are inaccessible due to proprietary settings or temporary server issues [9], their development represents a broader trend toward automation in geotechnical design.[7] This report evaluates the architectural feasibility, prior developments, potential utility, and necessary technical enhancements required to build a robust, enterprise-grade AI agent for PLAXIS.

Semantic Distinctions and the Evolution of Geotechnical Automation

In the geotechnical industry, the term "agent" has historically carried a commercial meaning. Regional engineering firms, such as Terrasol in France, have traditionally acted as commercial "PLAXIS agents," distributing licenses, managing entitlements, and providing software training to practicing engineers.[11] In the modern computational context, however, an "agent" refers to an autonomous, LLM-driven software entity designed to execute multi-step reasoning, tool usage, planning, and self-evaluation.[1]

Traditional automation in finite element modeling relies on linear scripting.[6] Since the release of its remote scripting application programming interface (API), PLAXIS has supported an HTTP-based REST API wrapper that communicates with an internal server running on a local port.[6] This allows users to write sequential Python scripts to build geometries, assign materials, generate meshes, and execute calculations.[15]

Despite these capabilities, traditional scripting exhibits severe limitations. It requires explicit,

hard-coded geometry commands and lacks the adaptability to handle varying inputs without extensive manual refactoring.[1] If a borehole log changes or a retaining wall is shifted, the entire script must be rewritten.[6]

An AI-driven agentic framework shifts this paradigm. Rather than merely executing predefined instructions, an AI agent leverages large language models (LLMs) to interpret unstructured inputs—such as natural language prompts, hand-drawn sketches, or PDF borehole logs—and dynamically generates, tests, and refactors the corresponding PLAXIS API code.[1] By utilizing tool-calling loops and self-evaluation mechanisms, the agent can adapt to geometric anomalies, run parametric optimization models, and handle complex multi-step reasoning tasks.[1]

Analysis of Prior Art and Existing Automation Frameworks

The development of an agentic system for PLAXIS builds upon several foundational frameworks and custom automation scripts developed by engineering firms and platform developers.

Custom Scripting Wrappers

Arup developed *AutoPlx*, a Python framework designed to run consecutive PLAXIS analyses by reading parameters from structured Excel sheets.[6] This tool can run in the background, on remote desktops, or overnight, allowing engineers to focus on other tasks while sequential analyses execute.[6]

Similarly, the *plxpy* library was created as a high-level Python wrapper to limit repetitive manual tasks and streamline geometry creation and structural capacity evaluation for complex tunnel linings at the Euston Crossover project.[6] During the Euston Approach project, this wrapper automated modeling for sequential stages, such as:

1. Pilot relaxation [6]
2. Pilot liner installation [6]
3. Further relaxation and liner aging [6]
4. Enlargement relaxation [6]
5. Enlargement liner installation [6]
6. Further relaxation and liner aging [6]

The wrapper extracted bending moments, shear forces, and displacements into comma-separated values (.csv) files, which were subsequently processed in PowerBI to calculate envelopes of maximum and minimum results for each cross section.[6] While highly functional, these tools remain fundamentally static; they cannot interpret natural language or

resolve unexpected runtime geometry errors without human intervention.[1]

Cloud-Based Parametric Platforms

The *VIKTOR platform* provides pre-built digital building blocks for PLAXIS automation.[19] For example, the sample-plaxis application parametrically defines road embankments and soil layouts via a structured web interface, executes the FEA in the cloud, and visualizes the results.[20] This centralizes the model-generation logic but still relies on standard user interface forms rather than autonomous agentic reasoning.[19]

Academic and Specialized AI Agents

GeoSim.AI represents an integration of LLMs with geomechanical simulation software.[8] It utilizes a Retrieval-Augmented Generation (RAG) architecture connected to a specialized geomechanics knowledge base to translate natural language commands and image inputs into precise technical scripts.[8]

Furthermore, the public repository viktor-platform/opensees-ai-agent demonstrates a working template of an LLM-powered structural agent.[13] This agent uses the Instructor library to generate structured JSON outputs from user prompts, which are subsequently parsed into Python code to build, analyze, and optimize steel platform models in OpenSeesPy, showing displacement plots in interactive 3D.[13] It leverages vkt.Chat for user interaction, vkt.PlotlyView for rendering 3D scenes, and vkt.Storage for sharing state data between different views.[13]

Automation Framework	Core Mechanism	Input Formats	Adaptability	State and Error Handling
Traditional Scripting (plxscripting) [14]	Sequential Python execution over local localhost port.	Raw Python code.[14]	Extremely Low (Hard-coded commands).[6]	Manual debugger; scripts fail immediately on geometric conflicts.[6]
High-Level Wrappers (AutoPlx / plxpy) [6]	Excel-to-Python parsing; static parameters.	Structured Excel files, CSV databases.[6]	Low (Configured parameter ranges only).[6]	Hard-coded retry loops; manual code adjustments required for split geometries.[6]

Parametric Cloud Apps (VIKTOR sample-plaxis) [20]	Cloud GUI connected to backend PLAXIS solvers.	Structured UI form fields, databases.[20]	Medium (Parametric constraints).[20]	Validated inputs prevent runtime crashes, but cannot handle arbitrary design shifts.
Structural AI Agents (OpenSees AI Agent) [13]	LLM core with Structured Output (Instructor) and tool-calling.[13]	Natural language, conversational prompts.[13]	High (Dynamic geometry and optimization loops).[13]	Automated self-correction loops; logs parsed by LLM to fix syntax errors.[1]
Geotechnical AI Assistants (GeoSim.AI) [8]	LLM with RAG, Geomechanical Knowledge Base, and target code toolsets.[8]	Natural language, engineering diagrams, images.[8]	Extremely High (Borehole log to code translation).[8]	Iterative RAG prompts to verify code compliance with geomechanical constraints.[8]

Practical Utility of a Geotechnical AI Agent

A robust, fully realized AI agent for PLAXIS introduces substantial operational benefits to civil and tunnel engineering workflows. By shifting low-value, repetitive tasks to an autonomous execution layer, engineering teams can focus on risk mitigation and high-value design decisions.[22]

Automated Stratigraphy and Geometry Interpretation

Geotechnical models begin with borehole data and stratigraphy sketches.[16] An AI agent utilizing computer vision and advanced language understanding can parse hand-drawn excavation drawings, stratigraphy sketches, and borehole logs directly into precise Python code.[7] The agent automates the creation of soil layers within PLAXIS, eliminating transcription errors between soil reports and numerical model setup.[7]

Dynamic Parametric Optimization

During iterative design phases, the agent can act as an optimizer. For a retaining wall excavation, the user can prompt the agent to optimize the wall embedment depth and strut spacing to ensure the maximum lateral wall deflection does not exceed allowable structural

limits.[6] The agent translates this goal into an objective function, iteratively modifying the PLAXIS geometry, generating the mesh, executing the calculation, and reading the displacements until the constraints are met.[6] The mathematical objective can be represented as:

$$\min \quad V(d_w, s_p)$$

$$\text{subject to} \quad u_{\max}(d_w, s_p) \leq u_{\text{allowable}}$$

where $V(d_w, s_p)$ represents the volume of structural elements (such as concrete wall thickness or steel reinforcement) as a function of the design variables d_w (embedment depth) and s_p (strut spacing), and $u_{\max}$ represents the maximum displacement vector computed by the PLAXIS finite element solver.

Closed-Loop Post-Processing and Automated Reporting

Extracting calculation results is historically tedious.[6] The AI agent can automatically access the PLAXIS Output server via the remote API to retrieve internal forces, bending moments, shear forces, and soil displacements.[6] It can compile these data points into structured CSV files, dynamically generate thrust-moment (T-M) interaction diagrams for concrete elements, and push the processed results directly into dynamic visualization dashboards like PowerBI.[6]

Technical Challenges and Areas for Improvement

To build or improve an automated geotechnical agent into a highly reliable tool, several domain-specific technical bottlenecks must be addressed.

The Post-Mesh Geometry Renaming Challenge

One of the most significant complications in automating PLAXIS via its API is how the software handles geometric object names.[6] When a geometry is defined (e.g., Polygon_1), PLAXIS assigns a standard index name.[6] However, when geometries intersect or when the model is meshed, PLAXIS splits these objects and dynamically assigns derived names (e.g., Polygon_1_1 or Polygon_1_Polygon_2).[6] Static scripts frequently fail during post-meshing phases because the original names no longer exist in the model state database.[6]

The AI agent must not rely on hard-coded name strings. It must be equipped with a spatial query tool that retrieves object references based on coordinate bounding boxes. By querying the active model explorer at coordinate (x, y), the agent can dynamically retrieve the active, split object ID before attempting to assign materials or boundaries.[6]

Python Environment and Runtime Conflicts

PLAXIS relies on a specialized, built-in Python distribution historically locked to older runtimes (such as Python 3.7 or 3.8) to maintain internal security and API stability.[24] The default interpreter path is typically:

C:\ProgramData\Bentley\Geotechnical\PLAXIS Python Distribution V2\python\python.exe [24]

Conversely, modern AI framework libraries (such as LangChain, CrewAI, AutoGen, and OpenAI's SDKs) require modern runtimes (Python 3.10 or higher). Running the AI agent and the PLAXIS remote connection within a single Python process is highly impractical due to dependency conflicts.[24]

To resolve this conflict, developers must implement an isolated, asynchronous client-server architecture. The modern AI agent should run in a containerized Python 3.11+ environment, communicating via gRPC or secure WebSockets with a lightweight "executor bridge" script running inside the local PLAXIS Python 3.7/3.8 interpreter.[24]

Stateful Remote Server and Port Management

The PLAXIS remote scripting interface requires launching the desktop application executable (Plaxis2DXInput.exe or Plaxis3DInput.exe), configuring a secure port, and assigning a unique encryption password via command-line arguments.[14] If the agent crashes, or if the session is not closed cleanly using the s_i.close() and s_o.close() methods, the communication port remains blocked, and the PLAXIS license may remain checked out on the network server.[14]

Developers must implement strict context managers within the agent's execution code. This ensures that the PLAXIS application process is monitored, and licenses are automatically released under a finally block or when a SIGTERM signal is received.[17]

Determinism and Hallucination Mitigation in Code Generation

LLMs are probabilistic and prone to syntax hallucinations.[13] In geotechnical FEA, a single incorrect syntax command or a self-intersecting polygon coordinates array can crash the PLAXIS calculation kernel or produce structurally unsafe calculations that fail to converge.[15] While Bentley has introduced a command autocomplete and command helper feature for Geotechnical SELECT Entitlement users to assist with manual command entries, the agent must employ separate programmatic validation methods to ensure stability [26]:

- **Structured Output Frameworks:** Use libraries like Pydantic and Instructor to force the LLM to output predictable JSON models containing coordinate arrays and predefined command options.[13]
- **Pre-Flight Validation Engine:** Before executing commands on the live PLAXIS port, a symbolic parsing engine should run the generated script through a validation check to ensure coordinates form closed, non-self-intersecting polygons.[16]
- **Self-Correction Feedback Loops:** If PLAXIS throws an execution error, the agent should

capture the stack trace from the server logs, feed it back to the LLM core, and prompt an autonomous correction cycle.[1]

Licensing, Entitlements, and Open-Source Compliance

Deploying a PLAXIS automation tool requires navigating complex licensing frameworks and strict open-source software terms.

Bentley Entitlements and Server Access

To activate and use the Remote Scripting server within PLAXIS Input or Output, the user must have a valid Bentley Geotechnical SELECT Entitlement (GSE, formerly PLAXIS-VIP) license.[14] Since January 1, 2022, traditional Hardlock (HWL) versions and VIP subscriptions have been discontinued, meaning remote scripting services are only supported on PLAXIS CONNECT Edition versions utilizing the Subscription Entitlement Service.[14] The server must be manually configured under the "Expert > Configure remote scripting server" menu to enable the local TCP connection port and password encryption.[14]

Parameter	Input Scripting Server	Output Scripting Server
Default TCP Port	10000 [14]	First available free port (typically 10001) [14]
Associated Executable	Plaxis2DXInput.exe / Plaxis3DInput.exe [17]	Plaxis2DXOutput.exe / Plaxis3DOutput.exe [14]
Primary Variable Binding	s_i (server), g_i (global model object) [14]	s_o (server), g_o (global model object) [14]
Command-Line Arguments	--AppServerPort, --AppServerPassword [17]	Dynamic (initiated via g_i.view()) [17]
Licensing Requirement	Bentley Geotechnical SELECT Entitlement (GSE) [14]	Bentley Geotechnical SELECT Entitlement (GSE) [14]

The Plaxis Public License (PPL) and Reciprocal Requirements

A critical legal constraint for anyone building a PLAXIS scripting tool or agent is the licensing of the core scripting library, plxscripting.[28] This library is distributed under the Plaxis Public License (PPL) Version 1.5, which is a modified version of the Reciprocal Public License (RPL).[28]

The PPL is built on the concept of strict reciprocity.[28] Under its terms, any modifications, derivative works, or required components (collectively defined as "Extensions") created by a user must be made available to the open-source community at large and Plaxis BV in particular when deployed in any form.[28] This deployment rule applies whether the software is shared with an outside party or deployed internally within an organization.[28]

Under the PPL, all components authored—including schemas, scripts, and source code, regardless of whether they are compiled into a single binary or split across a client-server application—must be shared publicly.[28] This reciprocal requirement represents a major legal consideration for corporate entities developing proprietary PLAXIS automation workflows, as they are legally obligated to release their source code to the community once the software is run.[28]

Strategic Architecture for an Advanced PLAXIS Agent

To achieve standard industry compliance and operational safety, a production-grade PLAXIS Agent should be designed around a multi-agent orchestration pattern utilizing specialized protocols.[2]

Multi-Agent Role Allocation

The system should distribute responsibilities among three specialized sub-agents [2]:

1. **The Geotechnical Geometry Agent:** Responsible for interpreting borehole coordinates, identifying materials (e.g., concrete strength grades, soil types), and generating the initial coordinate dictionary.[7]
2. **The Solver and Calculation Agent:** Handles phase configuration (e.g., greenfield state, stage-by-stage excavation, strut activation, and structural load application) and initiates the numerical calculations.[15]
3. **The Extraction and Validation Agent:** Connects to the PLAXIS Output server, monitors solver convergence, extracts structural forces, checks for stability failures, and formats report dashboards.[6]

Model Context Protocol (MCP) and Agent-to-Agent (A2A) Integration

By utilizing the Model Context Protocol (MCP) and standard Agent-to-Agent (A2A) communication, the PLAXIS Agent can collaborate with other engineering software agents.[2] For example, a Structural Agent (controlling CAD models in Revit) can negotiate wall thickness modifications directly with the Geotechnical Agent in real-time, matching structural capacity with soil stability constraints without manual file exchanges.[2]

Operational Recommendations and Development

Roadmap

To implement a highly reliable PLAXIS Agent, development should proceed through a series of structured engineering phases:

Phase 1: Establish a Decoupled Communication Bridge

To bypass the Python version conflicts between PLAXIS (Python 3.7/3.8) and modern AI libraries (Python 3.10+), develop an asynchronous client-server architecture.[24] The AI agent should run in a modern Python environment and send command requests over a secure local socket to a execution script running inside the PLAXIS-specific interpreter.[17]

Phase 2: Implement Spatial Query Tools

Create a helper library within the execution bridge that maps geometric objects based on physical coordinate bounds rather than static string names.[6] This spatial indexing ensures that when a geometry splits during meshing (e.g., Polygon_1 splitting into Polygon_1_1), the agent can query the database at coordinate (x, y) to retrieve the active, split object ID.[6]

Phase 3: Enforce Pydantic Validation and Structured Outputs

Integrate the Instructor library to parse user prompts into validated, structured JSON formats.[13] Define strict schemas for borehole geometries, wall dimensions, and calculation phases to ensure that generated values are within safe physical bounds before being executed by the PLAXIS command line solver.[13]

Phase 4: Configure Self-Correction Loops

Implement try-except blocks around all API calls.[15] If the PLAXIS solver throws a geometry clash or a convergence error, the agent must catch the exception stack trace, feed the code and the error back to the LLM core, and prompt the agent to autonomously generate a corrected code sequence.[1]

Phase 5: Ensure Open-Source Compliance

Align the software repository with the requirements of the Plaxis Public License (PPL).[28] Because any authored PLAXIS scripting wrappers deployed internally or externally must be shared with the open-source community, establish a public repository strategy that complies with PPL reciprocity while keeping proprietary corporate database connectors separated in an isolated microservice.[28]

Works cited

1. How to Build LLM Apps, AI Agents and Automated Workflows - DEV Community, accessed May 17, 2026, https://dev.to/yeahiasarker/ai-automation-how-to-build-llm-apps-ai-agents-and-

automated-workflows-5c1o
2. (PDF) Agentic Ecosystem in Engineering Design: A Framework for Interoperable Legacy Tools and Emergent Collaboration via MCP/A2A Protocols - ResearchGate, accessed May 17, 2026, https://www.researchgate.net/publication/390667861_Agentic_Ecosystem_in_Engineering_Design_A_Framework_for_Interoperable_Legacy_Tools_and_Emergent_Collaboration_via_MCPA2A_Protocols
3. Plaxis - Wikipedia, accessed May 17, 2026, https://en.wikipedia.org/wiki/Plaxis
4. PLAXIS 3D: Geotechnical Engineering Software | Bentley Systems, accessed May 17, 2026, https://www.bentley.com/software/plaxis-3d/
5. PLAXIS 2D: Geotechnical Engineering Software - Bentley Systems, accessed May 17, 2026, https://www.bentley.com/software/plaxis-2d/
6. Automation of soil-structure interaction finite element modelling using Python coding and the project BIM model | Major Projects Association, accessed May 17, 2026, https://majorprojects.org/resources/automation-of-soil-structure-interaction-finite-element-modelling-using-python-coding-and-the-project-bim-model/
7. Leveraging Generative AI for Automated Plaxis Model Creation - Prezi, accessed May 17, 2026, https://prezi.com/p/mkbyfxaze3fc/leveraging-generative-ai-for-automated-plaxis-model-creation/
8. GeoSim.AI: AI assistants for numerical simulations in geomechanics - arXiv, accessed May 17, 2026, https://arxiv.org/html/2501.14186v1
9. accessed January 1, 1970, https://github.com/souga15/plaxis-agent
10. accessed January 1, 1970, https://raw.githubusercontent.com/souga15/plaxis-agent/master/README.md
11. Terrasol Profile | Geoworld, accessed May 17, 2026, https://www.mygeoworld.com/company/terrasol
12. Plaxis Software, accessed May 17, 2026, https://www.civil.iitb.ac.in/~ajuneja/Plaxis%20program/OrderForm/PLAXIS-OrderForm.pdf
13. viktor-platform/opensees-ai-agent - GitHub, accessed May 17, 2026, https://github.com/viktor-platform/opensees-ai-agent
14. Using PLAXIS Remote scripting with the Python wrapper - Communities, accessed May 17, 2026, https://bentleysystems.service-now.com/community?id=kb_article&sysparm_article=KB0108870
15. API / Python scripting - Overview - PLAXIS - Communities, accessed May 17, 2026, https://bentleysystems.service-now.com/community?id=kb_article&sysparm_article=KB0107775
16. REST API and Python Integration - Bentley Software Documentation, accessed May 17, 2026, https://docs.bentley.com/LiveContent/web/PLAXIS%202D-v2025.1.3/Help/en/1_Manuals/Reference/topics/Appendices/REST_API_Essencials/REST_API_Python_label

.html
17. GeoStudio | PLAXIS - How to automate PLAXIS with Python using ..., accessed May 17, 2026, https://bentleysystems.service-now.com/community?id=kb_article_view&sysparm_article=KB0049677
18. LLM Workflows: From Automation to AI Agents (with Python) - YouTube, accessed May 17, 2026, https://www.youtube.com/watch?v=Nm_mmRTpWLg
19. Integrate PLAXIS models in parametric engineering tools with Python - VIKTOR.AI, accessed May 17, 2026, https://www.viktor.ai/blog/34/integration-plaxis-python-parametricdesign
20. VIKTOR.AI - GitHub, accessed May 17, 2026, https://github.com/viktor-platform
21. GitHub - viktor-platform/sample-plaxis: Parametrically define a road embankment and soil layout in PLAXIS using a VIKTOR application. Define materials in a database and use them in PLAXIS. Analyse the embankment using PLAXIS, and visualise the results in a separate view, accessed May 17, 2026, https://github.com/viktor-platform/sample-plaxis
22. Orchestrating Complex AI Workflows with AI Agents & LLMs - YouTube, accessed May 17, 2026, https://www.youtube.com/watch?v=OFq_CvRCpA0
23. 1- Easy PLAXIS 3D tutorial for beginners , interface - YouTube, accessed May 17, 2026, https://www.youtube.com/watch?v=prK3nBVVPOg
24. Start Using Python to Automate PLAXIS - VIKTOR.AI, accessed May 17, 2026, https://www.viktor.ai/blog/88/start-using-python-to-automate-plaxis
25. Does not work with Python 3.7 · Issue #737 · TomSchimansky/CustomTkinter - GitHub, accessed May 17, 2026, https://github.com/TomSchimansky/CustomTkinter/issues/737
26. PLAXIS 2D Autocomplete - YouTube, accessed May 17, 2026, https://www.youtube.com/watch?v=BwznS61j5ul
27. PLAXIS - Communities, accessed May 17, 2026, https://bentleysystems.service-now.com/community?id=kb_article&sysparm_article=KB0108794
28. plxscripting/LICENSE at main - GitHub, accessed May 17, 2026, https://github.com/cemsbv/plxscripting/blob/main/LICENSE